Lecture 21 → See the Network: CLI Tools, Wireshark, and pyshark

Why linux tools

- Network engineers diagnose and monitor networks using Linux and BSD tools
- Juniper JunOS (backbone routers) runs on FreeBSD
- Arista EOS (data center switches) runs on Linux
- pfSense / OPNsense (firewalls) run on FreeBSD
- tcpdump, ping, traceroute, ss are standard on all of them

Setup: multipass VM

Install Multipass:

```
# macOS
brew install multipass

# Windows: download installer from multipass.run

# Linux
sudo snap install multipass
```

Create and provision the VM (one time, 10–15 min):

```
multipass launch --name ec441 --cpus 2 --memory 4G --disk 20G \
  --cloud-init ec441_setup.yaml 24.04
```

Open a shell:

```
multipass shell ec441
```

Ping → ICMP echo

```
ping -c 5 8.8.8.8
ping -i 0.2 -c 20 8.8.8.8 # fast ping, watch RTT variance
```

Tracroute → path discovery

```
traceroute -m 20 8.8.8.8
traceroute -T -p 80 8.8.8.8 # TCP mode, bypasses ICMP filters
```

IP → interfaces, routes, ARP

```
ip addr show # interfaces and addresses
ip route show # forwarding table
ip neigh show # ARP cache (IP -> MAC)
```

SS → live TCP state

```
ss -t # established TCP connections
ss -tipm # full TCP internals
# Watch cwnd grow during iperf:
iperf3 -s &
iperf3 -c 127.0.0.1 -t 30 &
watch -n 0.5 'ss -tipm | grep iperf'
```

`ss -tipm` **key fields:**

Field	Meaning
`cwnd:N`	Congestion window (MSS units)
`ssthresh:N`	Slow-start threshold
`rtt:X/Y`	Smoothed RTT / RTTVAR (ms)
`retrans:N/M`	Current / total retransmissions

dig → DNS

```
dig google.com # full output
dig +short google.com # just the IP
dig @8.8.8.8 google.com # specify resolver
dig +trace google.com # full resolution chain
dig google.com MX # mail records
```

tcpdump → CLI capture

```
# Watch DNS queries live
sudo tcpdump -i any -n port 53
# Capture to file:
sudo tcpdump -i any -n -w ~/capture.pcap &
ping -c 5 8.8.8.8
dig google.com
curl -s https://example.com > /dev/null
sudo pkill tcpdump
```

Setup → wireshark

Platform	Install
macOS	`brew install --cask wireshark` or download from `wireshark.org`
Windows	Download installer from `wireshark.org`
Linux	`sudo apt-get install wireshark`

Transfer the capture

```
multipass transfer ec441:/home/ubuntu/capture.pcap ~/capture.pcap
```

From GUI to code

Wireshark

- One conversation at a time
- Click to explore
- Great for diagnosis

pyshark

- Treat a capture as a dataset
- Ask questions across thousands of packets
- Automate analysis, generate plots

<u>pyshark → load and iterate</u>

```
import pyshark
cap = pyshark.FileCapture('/home/ubuntu/capture.pcap')
for pkt in cap:
    print(pkt.highest_layer, pkt.length)
```

<u>pyshark → inspect a packet</u>

```
cap = pyshark.FileCapture('/home/ubuntu/capture.pcap')
pkt = next(iter(cap))
print(pkt.layers) # all layers present
print(pkt.ip.src) # source IP
print(pkt.ip.ttl) # TTL from L17
print(pkt.transport_layer) # TCP or UDP
```

<u>pyshark → extract DNS queries</u>

```
cap = pyshark.FileCapture('capture.pcap', display_filter='dns')
queries = []
for pkt in cap:
    try:
        if hasattr(pkt.dns, 'qry_name'):
            queries.append(pkt.dns.qry_name)
    except AttributeError:
        pass
for q in sorted(set(queries)):
    print(q)
```

<u>pyshark → plot TCP RTT</u>

```
import pyshark, matplotlib.pyplot as plt
cap = pyshark.FileCapture('capture.pcap',
       display_filter='tcp.analysis.ack_rtt')
times, rtts = [], []
for pkt in cap:
    try:
        times.append(float(pkt.sniff_timestamp))
        rtts.append(float(pkt.tcp.analysis_ack_rtt) * 1000)
    except AttributeError:
        pass
plt.plot(times, rtts, '.', markersize=3)
plt.xlabel('Time␣(s)')
plt.ylabel('RTT␣(ms)')
plt.savefig('rtt_plot.png')
```